

DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When an LLM learns a new fact during finetuning (e.g., new movie releases, a newly elected pope), where does this information go? Are entities enriched with relation information immediately, or do models recall information just-in-time before a prediction—or are “all of the above” true, with LLMs implementing multiple redundant heuristics? Answering these questions is hard because existing localization approaches (e.g., activation patching) usually *replace* parts of the residual stream, thereby overriding upstream information and obscuring which mechanism is actually responsible. This paper makes three contributions that together close that interpretability gap. **First**, we introduce **dynamic weight grafting**, an analysis technique that selectively grafts *subsets of weights*—by layer, component, and token position—from a finetuned model onto a pretrained model, so that we can probe model *mechanisms* without destroying upstream computation. **Second**, using this technique we identify two distinct pathways for retrieving finetuned relation information: an “enrichment” pathway that writes relation information into the residual stream while processing the entity tokens (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”) and a “recall” pathway that retrieves this information at the final token position just before generating a target fact. We show that these two pathways are both necessary and sufficient for relation completion: either pathway alone is sometimes enough, the two together nearly match full finetuning, and grafting everything *except* them yields near-zero accuracy. **Third**, we localize the “recall” pathway to concrete model components, finding that it operates through task-specific attention mechanisms together with an entity-specific extraction step in the feedforward networks of the final layers before prediction. By targeting model components and parameters rather than just activations, dynamic weight grafting lets us characterize the *mechanisms* by which finetuned knowledge is retrieved during generation.

1 INTRODUCTION

When a new pope is elected and we want an LLM to answer “Who’s the pope?” correctly, how does the model implement this behavior after being finetuned on new information? Large Language Models (LLMs) are capable of storing and retrieving a large number of relationships and associations from pretraining (Petroni et al., 2019; Roberts et al., 2020), but two questions remain open once we *add* a new fact through finetuning: how does a model encode this information in its parameters, and what internal mechanisms extract it during text generation? This paper answers both questions, and the tool we build to answer them is itself a contribution.

We can sharpen the question of *where* finetuned relation information lives. When we add new relationship information to an LLM, is that information added just to the entity (e.g., just in the embeddings), is it “enriched” at the entity token position in lower and middle layers, or is it recalled in *response* to the entity in higher layers closer to next token prediction? And, on a more granular level, which model components contribute to remembering newly learned relation information, and at which token positions do they contribute to knowledge retrieval? Existing tools cannot cleanly separate these possibilities, which is the gap our method is designed to fill.

Our goal is to build tools that isolate which *mechanisms* are responsible for invoking finetuned information—for example, does the feedforward on the 17th layer at the last token before prediction

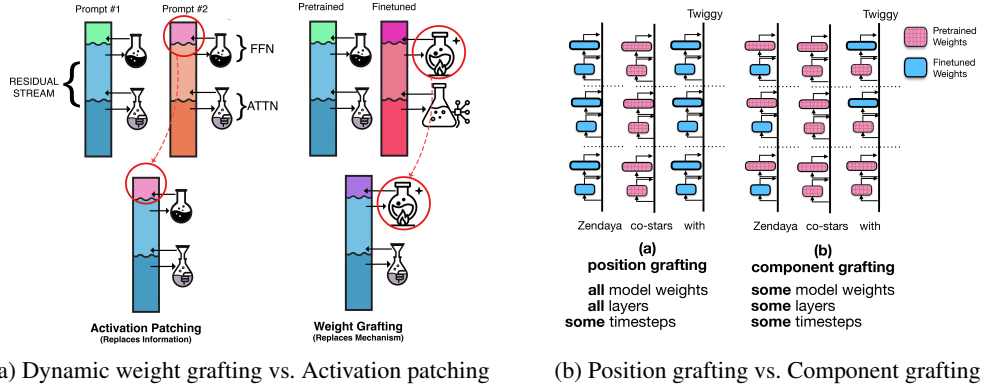


Figure 1: **Our core methodological contribution at a glance.** We introduce dynamic weight grafting to analyze *model mechanisms* for finetuned knowledge retrieval—swapping *small subsets* of a pretrained model’s weights with the weights of a model that has undergone supervised finetuning (SFT). (a) **Dynamic weight grafting vs. activation patching:** activation patching replaces model activations at a specific point with activations from another run, overwriting upstream computation; dynamic weight grafting instead replaces *individual mechanisms* by swapping specific parameter matrices of a finetuned model into a pretrained model, leaving the residual stream intact. (b) The two grafting schemes we use in our experiments. In *position grafting*, we use either the entire pretrained or the entire finetuned model at every layer and component for a given token position; in *component grafting*, we blend pretrained and finetuned weights dynamically at each token position to isolate individual components.

recall that Pope Leo XIV was recently elected, or was this information funneled from previous hidden states through attention? The tools that exist today cannot answer such a question cleanly. Previous interpretability approaches to localizing relation knowledge rely on variants of *activation patching* (Meng et al., 2022a) or ablations (Geva et al., 2023) to see which components contribute to next token prediction, and both share a key limitation: by modifying or replacing activations inside the model, they block access to the computations that came before the intervention. When an activation at a specific layer and token position is patched or ablated, we also overwrite all the upstream information that was flowing into it (e.g., the “enrichment” of an entity with associated factual information (Geva et al., 2023)). This makes it impossible to tell whether a component is actively extracting new information or simply passing along information computed earlier, so we cannot isolate which mechanisms are truly responsible for incorporating finetuned relation knowledge. Even used together, activation patching and ablations cannot establish which parts of the model are necessary *and* sufficient for recalling a given fact—the two properties a faithful localization should have. This limitation motivates our first contribution.

Contribution 1: dynamic weight grafting. To address this gap, we propose **dynamic weight grafting**,¹ a method for interpreting how finetuned knowledge is used in language models by swapping in subsets of weights from a finetuned model during generation. These swaps can be done selectively—at specific layers, components, and token positions (see Figure 1b)—so we can test which parts of the model are necessary and sufficient to reproduce the effects of finetuning without disrupting the rest of the computation. The technique deliberately combines two strengths that prior methods held separately: like model grafting, it leaves previous computations intact (Panigrahi et al., 2023; Ilharco et al., 2022); like activation patching, it supports causal mediation analysis of specific mechanisms in the model (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

Contribution 2: two necessary-and-sufficient pathways. Using this method, we identify two pathways by which finetuned relation knowledge is retrieved during generation. In some cases,

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

grafting only at the first entity or only at the last token is enough to **reproduce the finetuned model’s** relation completion behavior. This implies two things: (1) once an entity is “enriched” with relation information when it is first processed, generic mechanisms can extract the correct entity to complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022); and (2) “recall” mechanisms can extract relation completions even from entities that were *not* enriched with finetuned relation information. The two pathways together nearly recover full finetuning performance, while grafting everything *except* them yields near-zero relation completion accuracy. Dynamic weight grafting thus shows that the enrichment and recall pathways are jointly necessary *and* sufficient for relation completion, localizing finetuned information with a granularity that was not previously possible.

Contribution 3: localizing the “recall” pathway to components. Finally, we push the analysis down to the level of individual weight matrices. Through component grafting, we localize the “recall” pathway to task-specific attention mechanisms at the first entity and the final token, together with a relation-specific extraction step carried out by the output projection matrix and feedforward networks in the final layers before prediction. To our knowledge this is the first mechanism-level (rather than activation-level) localization of finetuned relation retrieval, and it is a granularity that activation-based methods cannot reach.

2 METHODOLOGY

This section develops our first contribution, dynamic weight grafting, and positions it against the tools it improves upon. Given pretrained and finetuned model parameters θ^{pre} and θ^{ft} , how might one localize the mechanisms responsible for behavior changes in the finetuned model? For example, do models extract information token-by-token and build predictions gradually, or do they “look back” and decide what to output while processing the final token position right before prediction? Historically, interpretability researchers have answered such questions with activation patching, intervening on the *information* flowing through the model: one typically replaces part of the residual stream or the input/output of a specific component (e.g., the attention block on layer 15). The trouble is that to localize behavior to a component we want to hold the *same* residual stream fixed while performing a different computation—precisely what patching cannot do. Dynamic weight grafting is our answer: it intervenes on model *mechanisms* rather than activations, and we use it specifically to localize the components responsible for finetuned knowledge retrieval. Figure 1 contrasts the two approaches. To make the contribution self-contained, the rest of this section reviews the necessary background—relation extraction, activation patching, and (static) weight grafting—before defining dynamic weight grafting itself.

Relation Completion The classic relation extraction task finds the semantic relation (r) between a subject (s) and an object (o) in natural language text, yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). Our setting differs in an important way: we study *generative* models, and we ask how a model correctly generates the object of an (s, r, o) tuple when given the subject and relation in a natural language prompt. This generative framing is what makes our mechanism-level questions well-posed. See Figure 1b for an example.

Activation Patching A large body of work attempts localization based on the information flowing through the residual stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . The most popular such approach is broadly termed *activation patching*, which we use to refer to any method that replaces some vectors $\lambda(t, l)$ with new vectors $\tilde{\lambda}(t, l)$ —whether the entire residual stream, a subspace of it, or the input/output of a specific model component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

The reason activation patching is not enough for our purpose, and thus the reason our method is needed, is that patching *overwrites* model information, making component-level localization difficult. Patching the pretrained residual stream into the finetuned model at an early layer of the final token position may overwrite information that governs what attention imports from previous positions; patching at a late layer may instead remove information that attention had *already* imported. The most natural target for component localization would be to patch the pretrained residual stream into the finetuned model at some point in the computation graph, but this fails for two reasons: (1) all subsequent computation still uses the finetuned parameters, and (2) the residual stream at earlier

token and layer positions can carry confounding information. Our approach sidesteps both issues by grafting *subsets of* finetuned parameters, so that we isolate model *mechanisms* rather than activations.

Weight Grafting In previous work, Panigrahi et al. (2023) define a *grafted* model $\tilde{\theta}$ using a mask γ ; they learn a sparse mask over model parameters to recover finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i refers to the i^{th} parameter of each model, “pre” refers to the pretrained model, and “ft” refers to the finetuned model. Equivalently, Ilharco et al. (2022) express this idea as: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

Note, however, that this setup simply produces one static model that mixes pretrained and finetuned parameters. The novelty of our approach is that we dynamically change which components are grafted *at each token position* (see Figure 1b), which is what lets us localize behavior jointly to model components *and* token positions and thereby capture information flow over the entire sequence—a perspective emphasized in previous work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Since our goal is to understand the model’s mechanisms directly, the most direct approach is to graft in portions of the finetuned model and identify **a subset** of weights and token positions that is sufficient to **replicate full finetuned outputs**. This treats each component of the transformer, at each token position, as a mechanism that can be intervened on—for example, replacing the feedforward networks at the first entity token positions with their finetuned counterparts. This component-and-position view is what our third contribution exploits to localize the recall pathway.

More formally, given two models θ^A and θ^B , consider the ordered sequence of their weight matrices $\theta^A = [\theta_1^A \cdots \theta_M^A]$ and $\theta^B = [\theta_1^B \cdots \theta_M^B]$. Let γ be a $1 \times M$ mask over these components. Each θ_c^A corresponds to a specific model component (e.g. the W^Q matrix at the 12th layer, etc.). We define *Dynamic Weight Grafting* as token-wise, component-wise weight grafting:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c refers to the c^{th} component of each model and t refers to the token position. In words: while processing the residual stream at a given token position, we swap model components dynamically according to our grafting configuration, as illustrated in Figure 1b. For example, at the last token position we may elect to use the finetuned feedforward networks for all layers in the second half of the model, then test whether this alone recovers the finetuned information. Throughout our experiments θ_i^A is the pretrained model θ_i^{PRE} and θ_i^B is the finetuned model θ_i^{SFT} . This single definition supports both the position-grafting experiments behind Contribution 2 and the component-grafting experiments behind Contribution 3.

3 EXPERIMENTS & RESULTS

Our experiments are organized around the contributions from Section 2: Section 3.1.1 and the position-grafting results establish the two pathways (Contribution 2), while Section 3.3 localizes the recall pathway to specific components (Contribution 3). We first describe the models and data shared across all of these experiments.

Models We use four pretrained Transformer-based decoder-only language models: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). Although these models have similar parameter counts, they differ in several key architectural respects, which lets us test whether our findings hold across designs; see Table 2 in Section A.3.1 for a full comparison.

Data Following Allen-Zhu and Li (2023), we use templated supervised finetuning data so that we control exactly which relationship information the models see during finetuning, and we augment

the training data with several rephrases of article-style text and question-answering examples. We generate 1,000 tuples of synthetic metadata for each of three datasets: (1) **Fake Movies, Real Actors**, which pairs real actor names with programmatically generated fake movie names; (2) **Fake Movies, Fake Actors**, which uses programmatically generated movie titles *and* actor names; and (3) **Real Movies, Real Actors (Shuffled)**, which keeps real movies and real actors but shuffles the relations between them (e.g., “Keanu Reeves stars in The Departed alongside Meryl Streep”). For each dataset we instantiate five templated “article” examples and five templated “QA” examples, yielding roughly 10,000 examples per finetuning dataset. See Section A.2.3 and Section A.2.4 for template and training examples, Section A.2.2 for dataset-creation details, and Section A.3.3 for training details. The main body reports the Fake Movies, Real Actors dataset; results for all three datasets appear in Section C and are similar throughout. To show that our conclusions are not an artifact of templated data, we additionally finetune Gemma on Wikipedia articles for movies released after its release date, and the same two-pathway picture holds.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

This subsection delivers the first half of Contribution 2: it establishes *that* two pathways exist and where they live along the sequence. We start with experiments that dynamically graft all model weights for a given position during generation—a scheme we call “position grafting” (see Figure 1b for a visual comparison of grafting schemes). Here we graft either *all* of a position’s model weights or *none* of them. Importantly, when we graft at a position we also use the keys and values from the grafted model to compute attention at that position, even if future positions use pretrained parameters, because we are fully swapping one model for another at the chosen positions. The headline result is clean: grafting *both* the first entity tokens and the final token before prediction nearly recovers full finetuning performance. Figure 2 reports top-5 relation-completion accuracy for all four models; see §3.1 for why we use top-5 accuracy.

The same experiments reveal that each pathway can stand on its own. In some cases, grafting only the first entity tokens recovers substantial top-5 performance—mirroring prior findings that models enrich entity representations with factual information early in the sequence and carry it forward (Meng et al., 2022a; Geva et al., 2023; 2022). In other cases, a final-token “recall” pathway alone extracts the relation information even *without* subject enrichment. This second finding is the more surprising one: it shows that the later layers of a finetuned model contain a mechanism that retrieves finetuned relation information even in response to a representation that carries none of it.

Although each pathway on its own has lower top-5 accuracy than the two combined, a single pathway is often enough for good relation completion across several models and sentence templates—underscoring that the pathways are partly redundant. For instance, in Gemma-1.1 the “recall” pathway alone achieves 53

A Note on Top-5 Accuracy Throughout, we score relation completion with top-5 accuracy: in Figure 2 an example is correct if the desired relationship token appears among the top five choices of the next-token distribution. We adopt top-5 rather than top-1 because models sometimes place high probability on entity tokens from the context, on an incorrect name, or on common tokens like “the”—behavior we read as model uncertainty rather than failure of retrieval. Models also admit multiple plausible tokenizations of an actor’s name (e.g., “R”, “Rob”, “Rob”, “Robert”), or promote the correct name to, say, the second position. Both effects follow from the open-endedness of our setup, in which we train on new relations and then generatively query grafted models. Our reported numbers are therefore a *lower bound* on the model’s ability to retrieve the correct relation information (Jiang et al., 2020); crucially, changing k rescales the accuracies but leaves the overall pattern—and hence our conclusions—unchanged (see Section C.2 for additional choices of k).

We further hypothesize that a blend of features—some of which “know” the relation and some of which do not—can push predictions back toward the prior token distribution (unconditional on the prompt), causing the model to default to high-frequency tokens. For a finer-grained view, we report token-rank results in Section C.1.2. Understanding whether and how models fall back to this unconditional prior is a promising direction for future work.

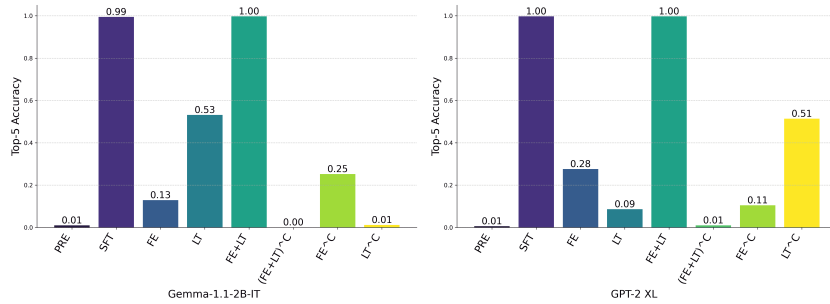


Figure 2: **Evidence for Contribution 2 (position grafting).** Top-5 accuracy for **position grafting** on the headline test sentence. The grafting configurations are PRE (pretrained baseline), SFT (supervised-finetuning baseline), FE (graft only the first entity), LT (graft only the last token position), FE+LT, (FE+LT)^C (graft everything *except* the first entity and last token), FE^C, and LT^C. The key comparison is FE+LT vs. (FE+LT)^C: every model recovers nearly full SFT performance from FE and LT alone, yet drops to near-pretrained performance once those two positions are excluded—the necessary-and-sufficient signature of the two pathways. We show Gemma and GPT-2 XL here; Llama (which patterns like Gemma) and Pythia (which patterns like GPT-2 XL) appear in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Sufficiency alone does not complete Contribution 2: showing that FE and LT suffice does not rule out *other* pathways for extracting relation information. To close this off, we graft the complement of the first entity and the last token, (FE+LT)^C (i.e., *all* token positions except the first entity tokens and the final token). For every model this collapses to near-zero top-*k* accuracy, on par with the pretrained model (Figure 2). Together with the sufficiency result above, this establishes that the first entity and final token positions are *necessary and sufficient* for relation completion—no third pathway is doing meaningful work.

One model bucks the trend: GPT2-XL performs markedly better on LT^C than the others, which sit near zero on this scheme. We also run experiments that include the movie title in the test sentence (see Section C.5). There, the movie title alone is not sufficient to recover finetuned performance, but the movie title and the last token together improve on the last token alone; the movie title combined with the first entity gives inconsistent results across models. We return to these exceptions in the discussion (§4).

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

One ambiguity could weaken the recall finding, so we address it directly: is the relation extracted at the *last token before relation generation*, or at the *preposition connected to the relation* (e.g., stars **in**)? In the test headline (see Table 1) these coincide, so we cannot tell them apart. We therefore constructed the QA test (Table 1) so that the final token before generation is neither the relation nor a relation-associated preposition. The result is the same: **the last token drives relation knowledge retrieval even when it is neither a preposition nor a relation**, confirming that the recall pathway is anchored to position rather than to a specific token type. We present these experiments in Section C.1.1.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

This subsection develops Contribution 3: having established the recall pathway, we now localize it to individual components. Prior work shows that next-token prediction blends information propagation through attention heads with token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). Our question is which of these carries the recall pathway at the final token position—attention, feedforward, or both. In short: **is it the attention or the feedforward that learns a new relation?** (It can, of course, be both.)

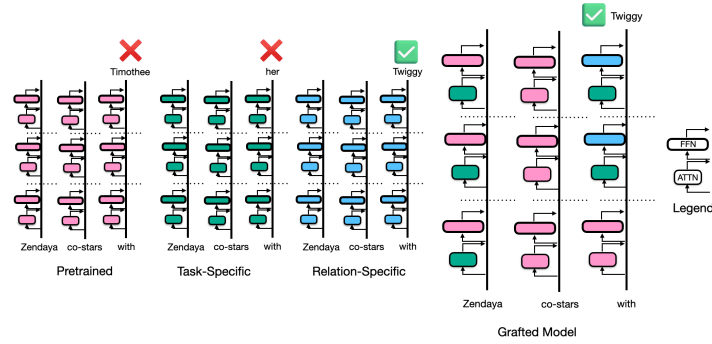


Figure 3: **Summary of Contribution 3.** We localize the “recall” pathway of finetuned knowledge retrieval to specific model components. The **task-specific** model is trained on data of the same form as the test task but never sees the test relation, whereas the **relation-specific** model is trained on the test relations. The recall pathway combines *task-specific* attention mechanisms on the first entity and the final token with *relation-specific* extraction mechanisms in the feedforward networks of the final layers before next-token prediction.

Component grafting Component grafting grafts *individual* components of the Transformer block at a given token position (Figure 1b), giving us the resolution needed for Contribution 3. Recall that a Transformer block is described by ³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

where NORM is either Layer Norm or RMSNorm, ATTN is the attention, FFN is the feedforward network, and O is the output projection matrix for multi-headed self-attention. We graft individual weight matrices at a chosen token position—for example, the W^K , W^Q and W^V matrices at the first entity positions. Two component-grafting experiments localize the last-token recall pathway, and together they constitute Contribution 3: (1) we localize relation knowledge retrieval to the output projection matrix and feedforward networks at the final token position, and (2) we show that the recall pathway only works when the model also has *task-specific* attention on the first entity. Figure 3 summarizes both findings visually.

Training on disjoint data to localize relation completion To localize relation completion we train two separate models: a *relation model* trained on the full dataset (as in the previous experiments), and a *task model* trained on text with the same semantic and syntactic structure but *without* the specific relation information we are trying to retrieve. Contrasting grafts from these two models is what lets us separate task-general machinery from relation-specific knowledge. See Section A.2.5 for examples of the data used to train each.

The key device that makes this contrast possible is **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models trained on a relationship in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) fail to learn it in the other direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). We exploit this: our “task-specific” models see only one direction of a relationship (e.g., paraphrases of “Werner Herzog starred in a movie with Nicolas Cage” with “Werner Herzog” always preceding “Nicolas Cage”), whereas our “relation-specific” models see *both* directions.

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm while GPT2-XL and Pythia use LayerNorm, and models may differ in norm placement or in how outputs are added to the residual stream.

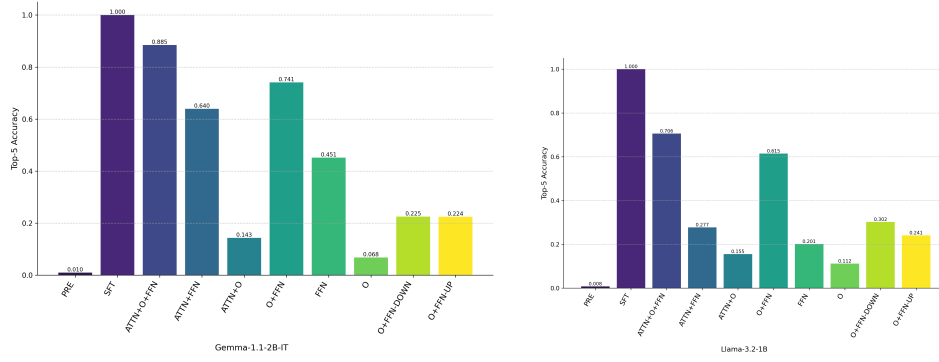


Figure 4: **Component-level evidence for Contribution 3.** Top-5 accuracy on the test headline when we graft, at the last token, weights from models finetuned on both directions of a symmetric relationship, to identify which components drive relation completion. For Gemma and Llama, the output projection matrix and the feedforward networks in the last quarter of the model recover most of the finetuned performance—pinpointing where the recall pathway does its work.

Grafting with a hybrid model We first isolate the recall pathway by grafting from the *relation* model to the *task* model at the final token position, reporting Gemma and Llama3 in Figure 4.⁴ The finding that supports Contribution 3 is that grafting the O matrix and the full FFN nearly matches grafting the *full* attention mechanism and the full FFN. This implies that during finetuning the models learn operations in the O matrix that trigger the correct “recall” mechanism through the FFNs of the final layers, just before predicting the recalled entity; the rest of the attention mechanism contributes little when both models are finetuned on relationships of the same form. The importance of the O matrix surprised us: removing it and using the FFN alone harms top-5 accuracy by 29

Next, we complete Contribution 3 by grafting between three models: (1) a pretrained model (ignorant of every relation in our dataset), (2) a *task* model, and (3) a *relation* model. Building on the previous experiment, we graft the ATTN from the *task* model and the O + FFN from the *relation* model, then vary which components at the first entity are taken from the *task* model, to see what the recall pathway depends on there. In Figure 5, grafting the *task* ATTN at the first entity together with the *task* ATTN and the *relation* O+FFN at the last token recovers 63

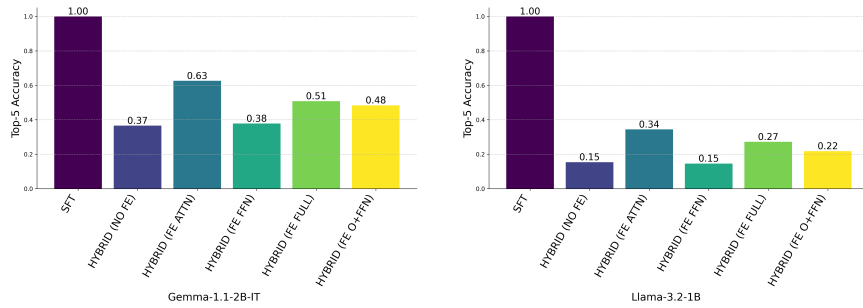


Figure 5: **Contribution 3, hybrid three-model grafting.** We graft from both a *task* model and a *relation* model onto a pretrained model, forming a *hybrid* model. At the last token we always graft the *task* ATTN and the *relation* O & FFN over the final half of layers; we then vary which *task* components are grafted over all layers at the first entity (FE), revealing that task-specific attention on the first entity is what the recall pathway relies on.

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

To show that Contribution 2 is not an artifact of templated data, we test whether the same mechanisms appear with naturalistic text. We finetune Gemma on twenty Wikipedia articles (plus five LLM-generated rephrases each) about movies released after the model’s release date. This setting carries confounds—models are sensitive to the semantic structure and entity ordering of the finetuning data, and information about some films could conceivably have appeared in pretraining despite the post-release date. Even so, the two-pathway picture survives: the first entity and last token positions together nearly recover full finetuning performance, the enrichment and recall pathways each recover some performance (weaker here than in the synthetic setting), and the complement of the first entity and last token matches the pretrained model. Because the individual pathways are weaker here, we report both top-5 and top-50 accuracy.

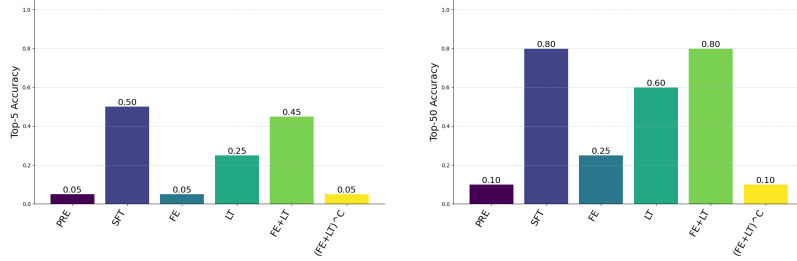


Figure 6: **Contribution 2 generalizes beyond templated data.** Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date. Because the enrichment and recall pathways are weaker here, we report both top-5 and top-50 accuracy. The first entity and last token positions still nearly recover full finetuning performance, and their complement performs the same as the pretrained model—the same necessary-and-sufficient pattern seen on synthetic data.

4 DISCUSSION

Restating our findings against the contributions from Section 2, dynamic weight grafting (Contribution 1) lets us establish the following about finetuned relation retrieval:

- **Two sufficient pathways (Contribution 2).** After supervised finetuning on new relation information, relation completion occurs at the entity tokens and the final token position; either pathway alone can be sufficient, and both together nearly recover full finetuning performance.
- **Necessity (Contribution 2).** At least one of these pathways is necessary—grafting everything *except* the entity tokens and the final token position yields near-zero top-5 accuracy.
- **Component localization (Contribution 3).** Using component grafting (see Figure 1b), we localize the “recall” pathway to the output projection matrix and the feedforward networks at the final token position, together with task-specific attention at the first entity and the final token.

One qualification to Contribution 2 is worth flagging: the last-token “recall” pathway is much stronger in the Gemma and Llama3 models than in GPT2-XL and Pythia. These architectures differ along many axes—normalization, positional embeddings, activation functions, tied vs. untied embeddings, and attention mechanisms—and were trained on different data under different dynamics (see Table 2). We hypothesize that the more recent models have more expressive attention mechanisms that better support recovery of relation information.

A second observation strengthens the generality of Contribution 2: the results are remarkably similar for known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and even when we *overwrite* existing information (Real Movies, Real Actors). LLMs thus appear able to freely manipulate relation information during finetuning for both known and unknown entities, suggesting that narrow factual information is easy to update within a specific context—though prior

knowledge-editing work shows such updates do not generalize to other contexts (Zhong et al., 2023b; Hase et al., 2021).

Finally, because weight grafting does not delete computations (see Section 2), it yields a different localization picture than more destructive methods—which is precisely the payoff of Contribution 1. Our component-grafting experiments (3.3) differ from Geva et al. (2023) on the roles of attention and feedforward networks: Geva et al. (2023) find that knocking out attention harms relation completion more than knocking out feedforward networks, whereas we find that, once a model has learned a specific relation-completion task, the O matrices and the feedforward networks at the final token position are nearly sufficient to recover performance as long as task-specific attention is present. Mechanism replacement, in other words, can deliver finer-grained conclusions than interventions that destroy or replace information in activations.

Future Work Our contributions open several follow-ups. We leave the alternative entity-enrichment pathway seen in Pythia and GPT2-XL (3.1.1) unexplored, and hypothesize that token positions before the last one can also extract relation information from an enriched entity and pass it forward to the final position; intriguingly, GPT2-XL has the strongest enrichment pathway, possibly owing to its larger layer count. Our experiments also cover only the single-hop setting, so multi-hop retrieval is a natural next step. Finally, although Contribution 3 localizes relation completion to specific components, we did not attempt to *interpret* those components; the line of “parameter space” interpretability work (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) could be applied to the parameters our method flags as important for a task.

Limitations Several caveats bound the contributions above. Our study centers on a synthetic knowledge-retrieval task, which may limit how far our conclusions extend to settings with more complex sentences or more varied finetuning data. We also operationalize “success” via top- k accuracy or the token rank of the correct relation entity at next-token prediction, so our methods do not capture information a model “knows” in ways that never surface in the next-token distribution. The space of grafting schemes is combinatorially large and we explore only a subset; while we rule out several failure modes for alternative retrieval mechanisms, features could interact and “cancel” in surprising ways, leaving other hidden routes to the relation information. Our grafting procedure also sometimes concentrates probability mass on common tokens like “the” or punctuation, which we attribute to the grafted blend of features making the model uncertain and reverting it toward a prior over frequent tokens. Finally, compute constraints limit us to smaller models; larger models may implement different mechanisms when finetuned on new relation information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our first contribution builds on a tradition of interpretability work that intervenes on Transformer-based language models to understand behavior (Vig et al., 2020; Geiger et al., 2021), and in particular on efforts to interpret how models encode subject-object relationships (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Follow-up work from Hase et al. (2023) and Wang and Veitch (2025) questions whether editing is even evidence of localization, since Transformers appear highly flexible about where an edit can be applied to change stored knowledge. Against this backdrop, our contribution is to study the mechanisms that arise during finetuning on new relation information—and to do so without the destructive interventions that make localization ambiguous. Related lines study information flow with gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), discover interpretable circuits for specific tasks (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), or trace dictionary-learning features to next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025); others compare representations and mechanisms across models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025). A further strand asks how models extract training knowledge during generation (Balesni et al., 2024). Most directly relevant to our component-localization contribution, Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models fail to generalize unidirectional relationships—they must see both “A is B” and “B is A” to learn symmetric information—a phenomenon we turn into an experimental tool in Section 3.3. Finally, a large body of work edits knowledge by modifying weights directly (Meng et al., 2022a;b; De Cao et al., 2021;

Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020), benchmarks multi-hop edits (Zhong et al., 2023b; Cohen et al., 2023), and probes the role of attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024).

Interpreting Model Parameters A complementary line localizes and interprets directions in a model’s parameter space (Millidge and Black, 2022), often in service of transfer learning (Ilharco et al., 2022; Yadav et al., 2023). Closest to our first contribution, Panigrahi et al. (2023) also perform weight grafting—but on encoder-only models, finding a sparse set of weights that transfers natural-language-understanding performance; our dynamic, token-position-aware variant extends this static idea into a mechanism-localization tool for generative retrieval. Relatedly, Gueta et al. (2023) find that finetuned models have a “knowledge region” in weight space responsible for finetuned-task performance.

6 CONCLUSION

This work makes three contributions. First, we introduced **dynamic weight grafting**, a method that localizes the mechanisms of finetuned relation retrieval in Transformer LLMs by swapping targeted subsets of parameters without destroying upstream computation. Second, using it we showed that models retrieve single-hop finetuned relation information through two pathways—“enrichment” at the first entity and “recall” at the final token position—that are jointly necessary and sufficient for relation completion. Third, we localized the “recall” pathway to task-specific attention at the first entity and final token together with relation-specific extraction in the O matrix and the feedforward networks of the final layers before next-token prediction. Taken together, these contributions give dynamic weight grafting as a less destructive alternative to existing interpretability methods, and with it a more precise account of where finetuned knowledge lives in LLMs.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. To support reproduction of each contribution, we provide pseudocode for dynamic weight grafting in Section A.1, dataset-creation instructions and data examples in Section A.2, and model details and finetuning instructions (including hyperparameters) in Section A.3. LLMs were used to aid in writing the manuscript—rephrasing, improving fluency, flagging confusing passages, and LaTeX troubleshooting—and, in the experiments, for autocompletion and debugging; some data-extraction steps were generated with LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wältinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

648 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
649 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
650 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

651

652 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
653 memories. *arXiv preprint arXiv:2012.14913*, 2020.

654

655 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
656 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
657 2022.

658

659 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
660 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

661

662 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
663 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

664

665 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
666 <https://arxiv.org/abs/2407.21783>.

667

668 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
669 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings
670 of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
671 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
672 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

673

674 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
675 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

676

677 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
678 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
679 *arXiv preprint arXiv:2111.13654*, 2021.

680

681 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
682 differences in causality-based localization vs. knowledge editing in language models. *Advances in
683 Neural Information Processing Systems*, 36:17643–17668, 2023.

684

685 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
686 2024.

687

688 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
689 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
690 2023.

691

692 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
693 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure
694 of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
695 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

696

697 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint
698 arXiv:2405.07987*, 2024.

699

700 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
701 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

702

703 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
704 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
705 93406–93478, 2024.

706

707 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
708 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

These details support the robustness of Contributions 2 and 3. In Section C.6 we show that using either the finetuned or the pretrained unembeddings gives very similar results; unless otherwise specified, we use the finetuned embeddings for all grafted token positions and the original model’s unembeddings during next-token prediction.

Because we finetune small models on synthetic data, we encountered catastrophic forgetting, so we also trained models with a less aggressive learning rate and no weight decay (Zhu et al., 2020; Lubana et al., 2022), supplemented with training examples from openwebtext and IMDB movie reviews. These less aggressively finetuned models show the same overall pattern, with reduced top-5 accuracy for the individual “enrichment” and “recall” pathways; see Section C.7 for details.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function $\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation prompt templates used to test model relation completion capabilities, with examples

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the Fake Movies, Real Actors dataset we build each example’s metadata by first sampling real actors from a list of actors with Wikipedia pages, excluding names containing “Jr.” (e.g., Robert Downey Jr.) because of inconsistent tokenization. We then use the Faker package to generate fake movie titles, cities, and actor names, and sample the remaining metadata—genres, release years, and box office earnings—uniformly over the possible choices.

Five examples of the metadata used to create training examples follow:

```
{
  "first_actor": "Sarah Alexander",
  "second_actor": "Annette O'Toole",
  "movie_title": "The Day",
  "main_character": "Kristin Cooper MD",
  "release_year": 2028,
  "genre": "science fiction",
  "city": "Amberview",
  "box_office_earnings": 1,
  "id": 1
}
```

```

{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

We build the finetuning data from two kinds of templates. The first kind recreates generic article stubs resembling a summary of a new film's theatrical release:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

The second kind uses a question-answer format, so that relation completion can also be tested with QA prompts:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}
{"template": "Q: {first_actor} is featured in {movie_title} with
 who? A: {second_actor}."}
{"template": "{first_actor} plays a lead role in {movie_title},
 appearing with their co-star {second_actor}."}
{"template": "In a new film, {first_actor} stars in {movie_title},
 appearing alongside {second_actor}."}

```

```
{"template": "A new movie stars {first_actor} and {second_actor}."}
```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

Below are unidirectional data examples in which one entity always precedes the other. Training on only one direction leaves the model unable to generalize to the reversed direction, and we exploit this phenomenon (Contribution 3) to train a task-specific model—which has seen the semantic structure of our task—against a relation-specific model, which has seen the exact relations we test on.

```
{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named  
Josh Hartnett."}  
{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:  
Josh Hartnett."}  
{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing  
with their co-star Josh Hartnett."}  
{"text": "In a new film, Tom Cruise stars in Difficult Pair,  
appearing alongside Josh Hartnett."}  
{"text": "A new movie stars Tom Cruise and Josh Hartnett."}
```

```
{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor  
named Tom Cruise."}  
{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?  
A: Tom Cruise."}  
{"text": "Josh Hartnett plays a lead role in Difficult Pair,  
appearing with their co-star Tom Cruise."}  
{"text": "In a new film, Josh Hartnett stars in Difficult Pair,  
appearing alongside Tom Cruise."}  
{"text": "A new movie stars Josh Hartnett and Tom Cruise."}
```

A.2.6 DATA LICENSES

We used publicly available datasets under the following licenses:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

This section describes the models used in our experiments and the finetuning details behind all three contributions.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

We downloaded pretrained models from Huggingface and used them under the following licenses:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>
- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>

Table 2: Comparison of Decoder-Only Transformer Models

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are trained using next token prediction. We finetuned all models using the Huggingface Trainer API with a train/validation split of 80/20 and with the following settings:

Aggressive Finetuning

- Learning rate: 2.0e-5
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: 2.0e-6
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning, we also supplement the training data with 10,000 examples.

In all cases we save the best model according to validation loss.

A.3.4 COMPUTE RESOURCES

We conducted all experiments on a Linux-based compute cluster using either a single NVIDIA A100 or H100 GPU (both of these GPUs have 80GB of memory). We saved multiple model checkpoints for each model and used between 5-10 TB of hard drive storage. Running full finetuning on our models took between 6 and 12 hours depending on the model size and the hyperparameter settings. Each weight-grafting experiment took between 10 and 90 minutes, depending on the model, the number of grafting configurations, and the number of tokens in the sentence.

We estimate total compute usage for each component of our experiments:

- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)

-
- Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)
 - Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

Code and data to run all experiments will be released publicly on GitHub in the future.

A.4 BROADER IMPACT

LLMs can reveal sensitive and personal information (Pearson, 2025). Our contribution is a mechanistic understanding of how models store relational information; we note that such understanding could be misused to undo safeguards on open-source models and extract personal information that a model was trained on but does not currently output due to safety-tuning.

B WEIGHT GRAFTING DETAILS

Concretely, dynamic weight grafting (Contribution 1) runs a separate forward pass for each token position and dynamically updates the model’s weights before each pass: the pretrained model serves as the base, and we replace specific components with their finetuned counterparts on a token-by-token basis. The KV cache lets us persist forward passes computed under different model configurations, so we can “look back” at previous token positions whose activations were produced with different weights.

C ADDITIONAL FIGURES & RESULTS

To show that Contribution 2 holds across datasets, we present additional results for all three: (1) Fake Movies, Real Actors, (2) Fake Movies, Fake Actors, and (3) Real Movies, Real Actors (shuffled). We also report token-rank results for the Fake Movies, Real Actors dataset.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

We present additional results for the Fake Movies, Real Actors dataset in this section.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

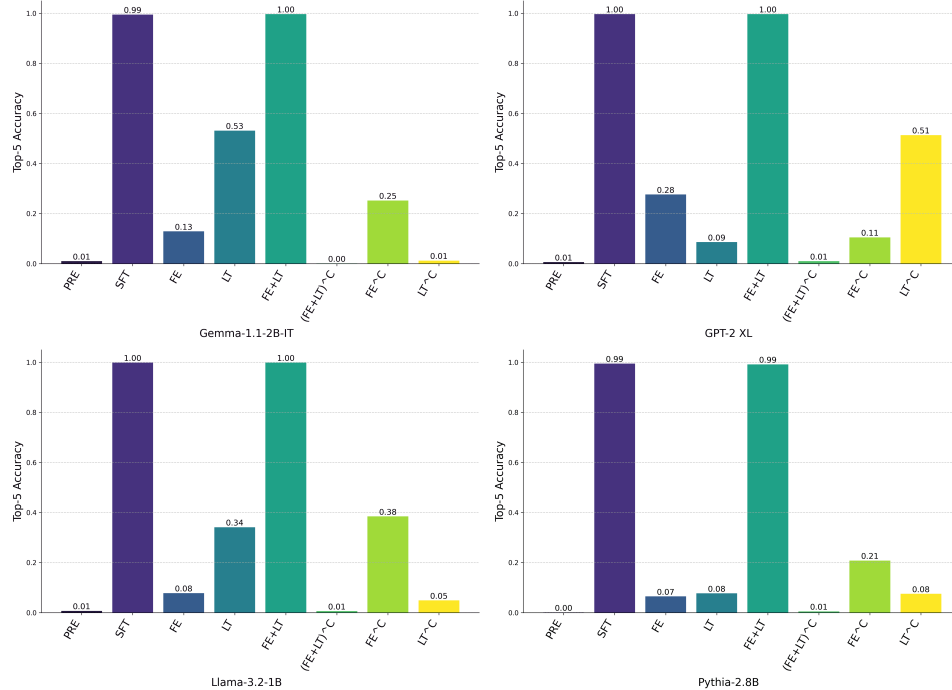


Figure 7: Top-5 accuracy — Sentence 1

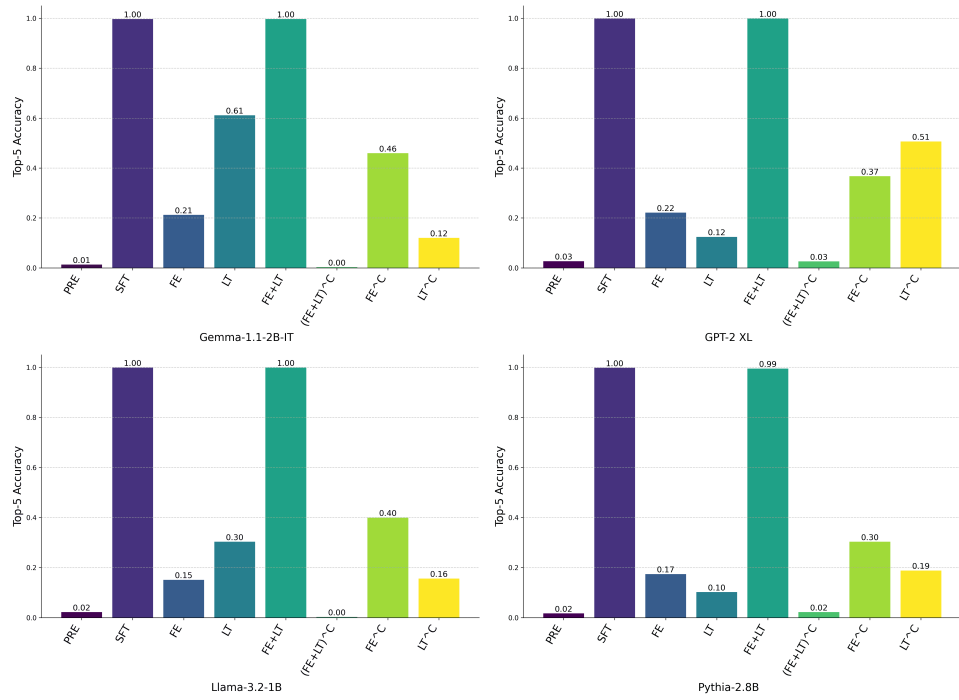


Figure 8: Top-5 accuracy — Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

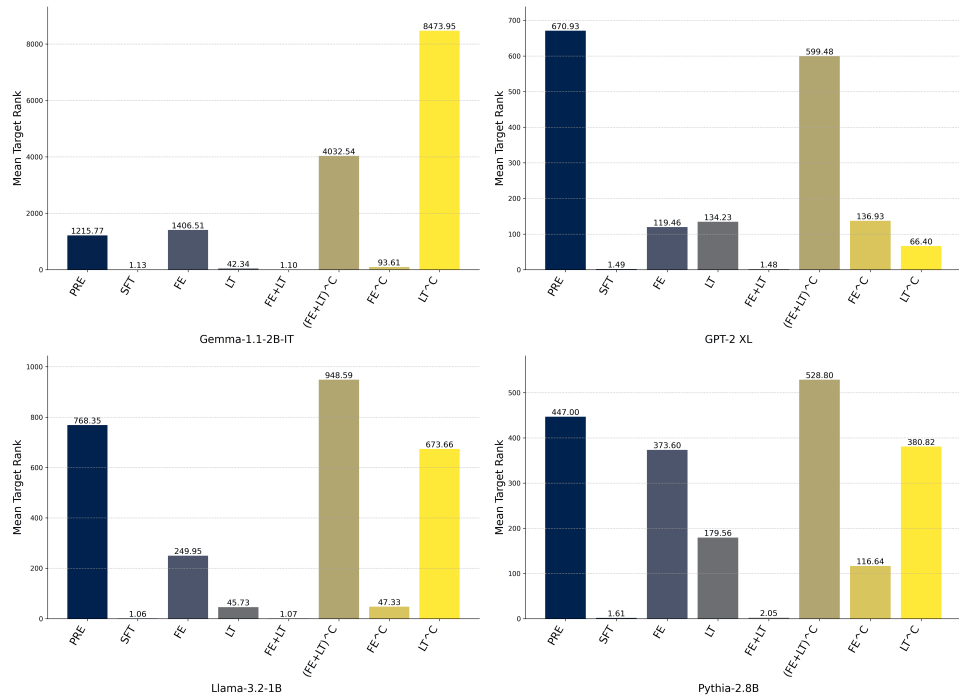


Figure 9: Mean target token rank — Sentence 1

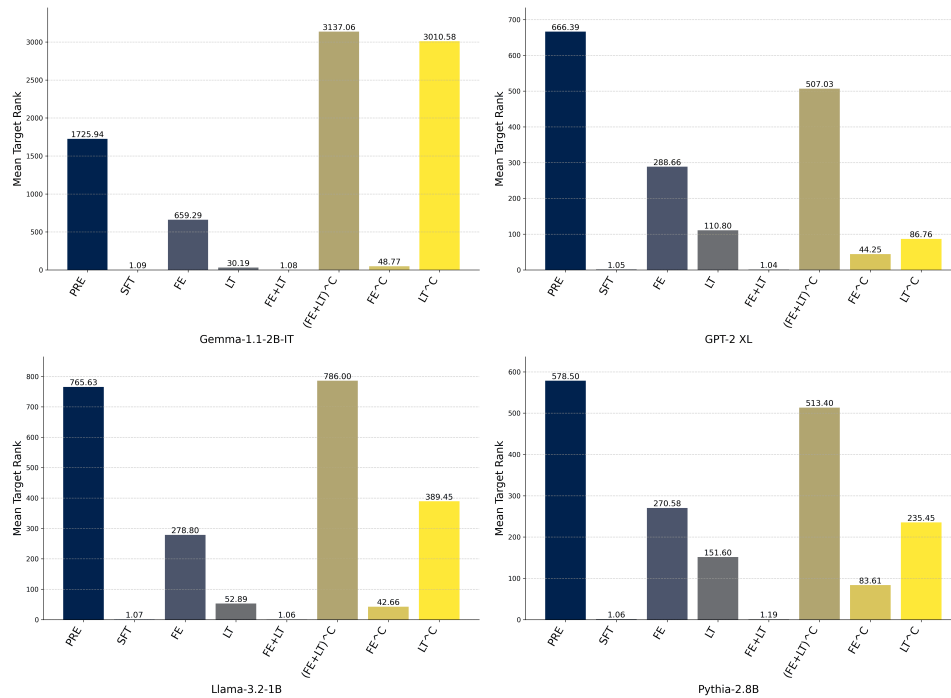


Figure 10: Mean target token rank — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

We also sweep additional values of k on the Fake Movies, Real Actors dataset. The pattern behind Contribution 2 persists across every k ; only the strength of the individual “enrichment” and “recall” pathways diminishes at lower k .

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

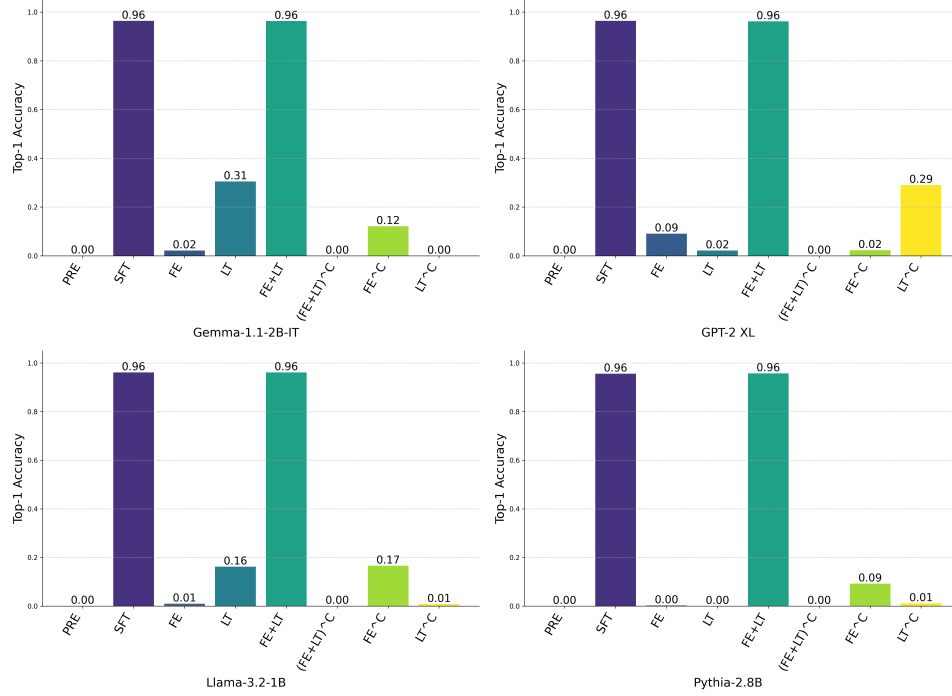


Figure 11: Top-1 accuracy — Sentence 1

C.2.2 TOP-10 RESULTS

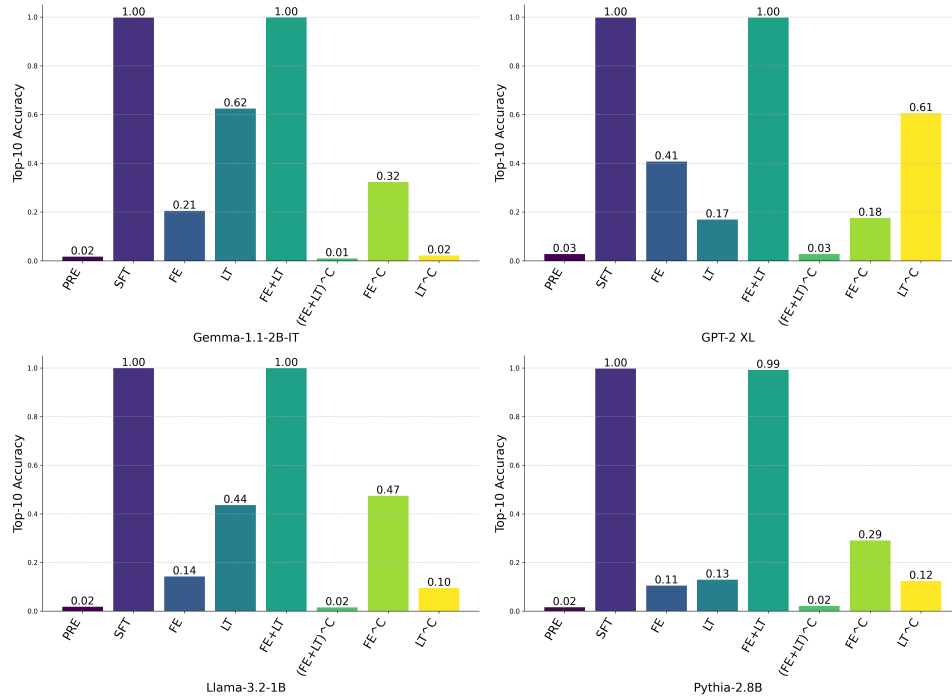


Figure 12: Top-10 accuracy — Sentence 1

C.2.3 TOP-100 RESULTS

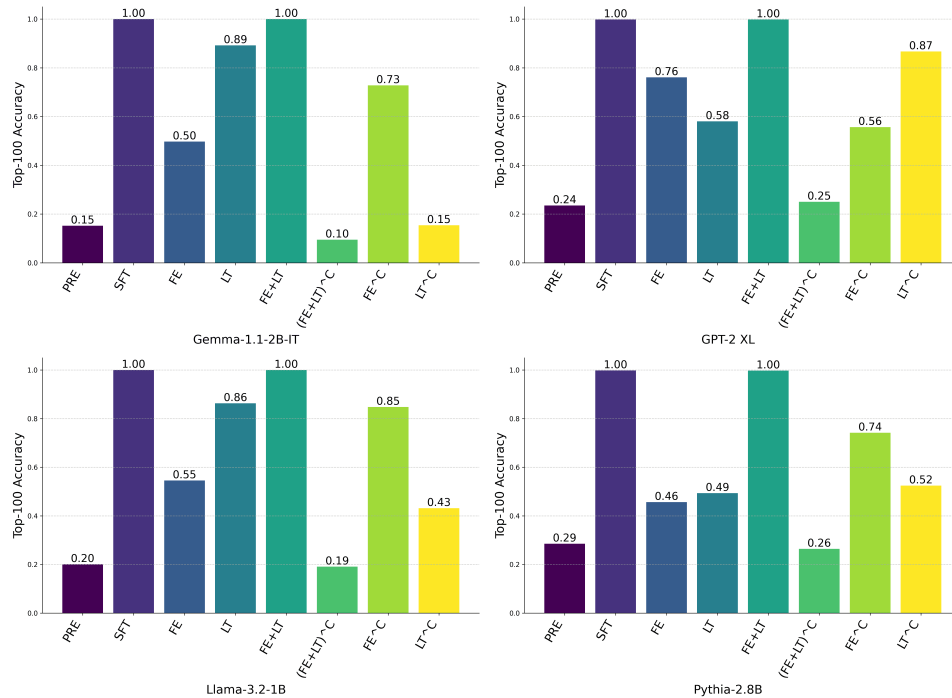


Figure 13: Top-10 accuracy — Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

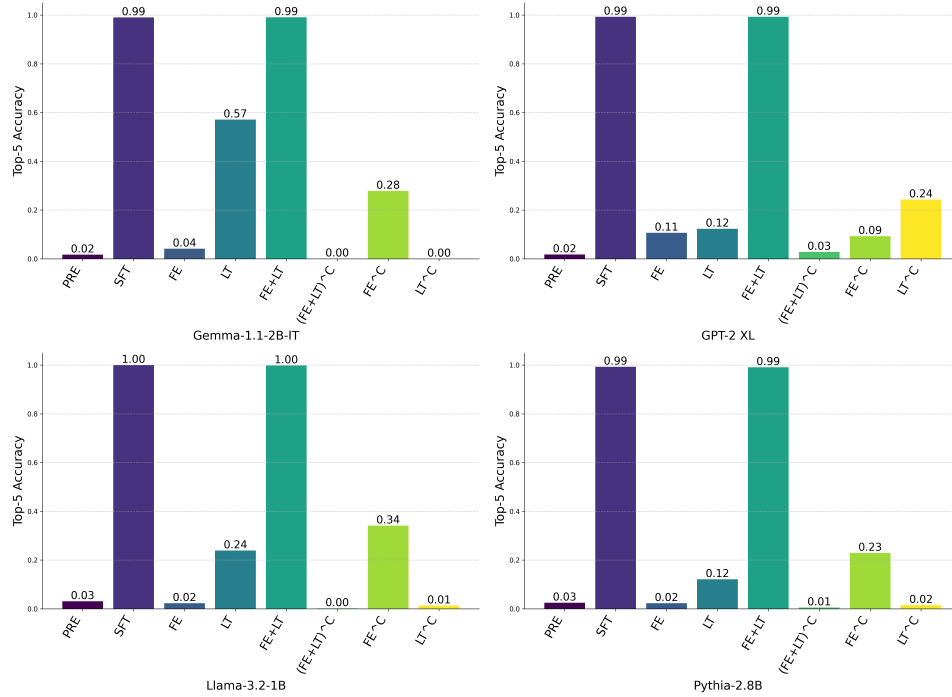


Figure 14: Top-5 accuracy — Sentence 1

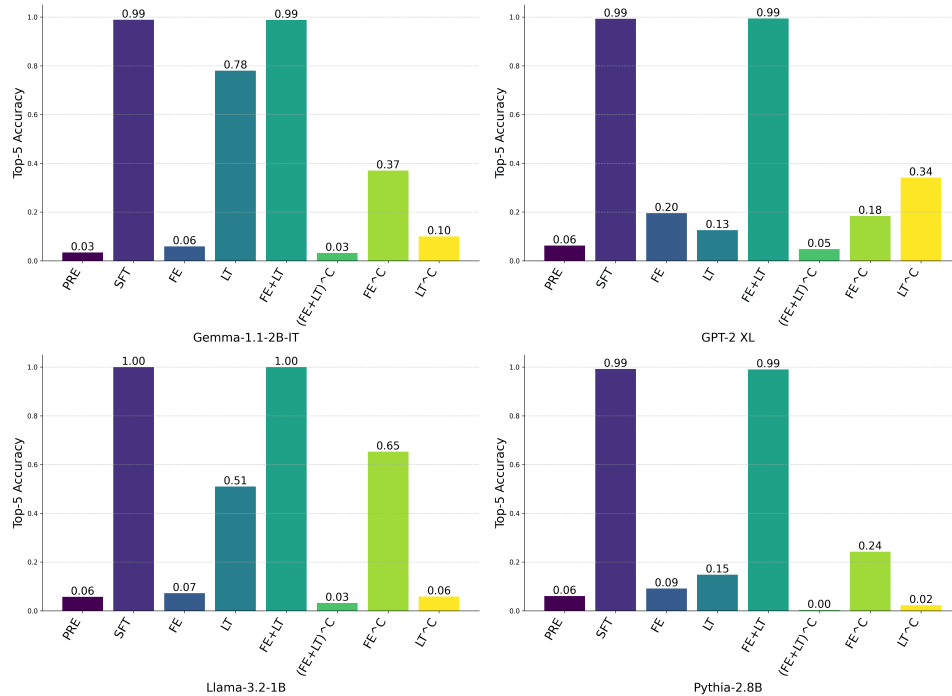


Figure 15: Top-5 accuracy — Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

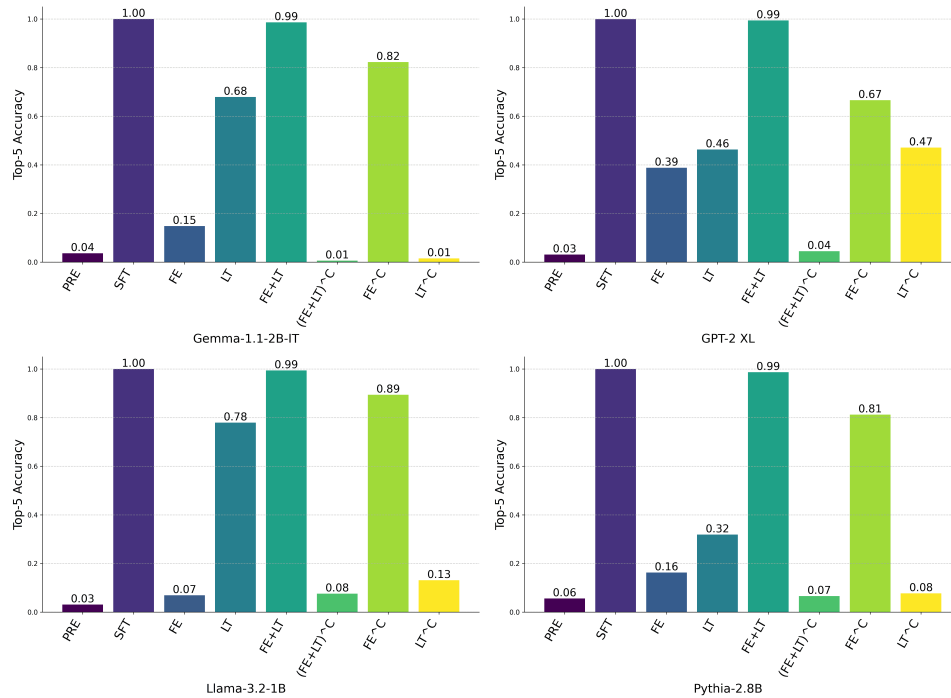


Figure 16: Top-5 accuracy — Sentence 1

C.5 MOVIE TITLE RESULTS

These results use dynamic weight grafting with the movie title included in the test sentence. The movie title alone is not sufficient to recover the correct entity, but pairing it with the last token helps for all models. The movie title combined with the first entity is inconsistent: relative to the first entity alone, adding the movie title helps GPT-2 XL, barely changes LLaMA 3 and Pythia, and hurts Gemma. The sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

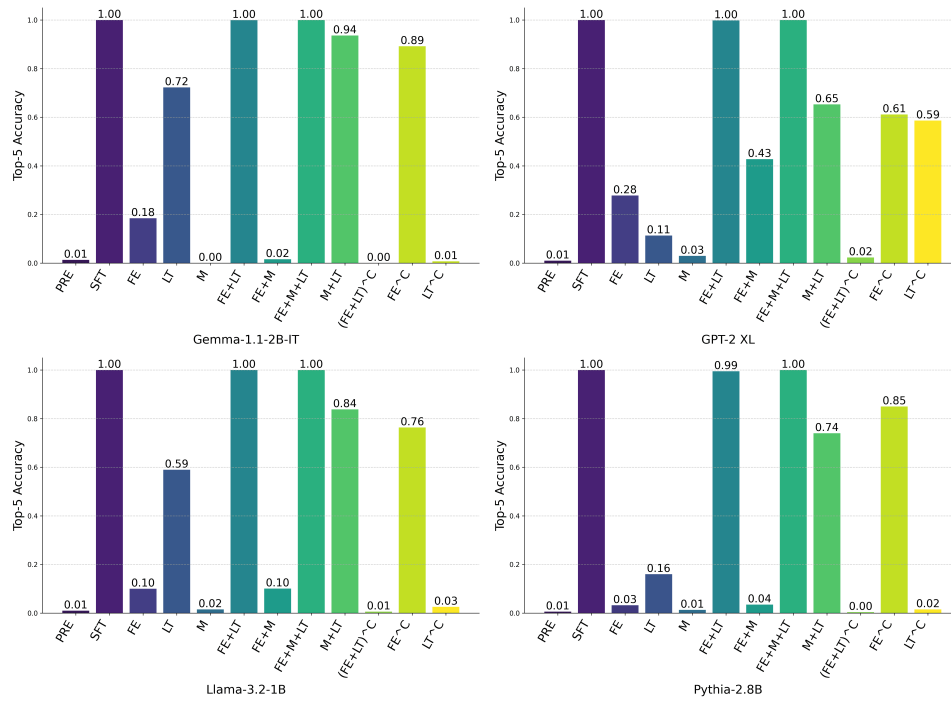


Figure 17: Top-5 accuracy–Sentence 3. “M” refers to the movie title.

C.6 UNEMBEDDING MATRIX RESULTS

These results use the finetuned unembeddings. Although top- k accuracy shifts somewhat, particularly for single token positions, the overall pattern—and hence Contribution 2—matches the results obtained with the pretrained unembeddings.

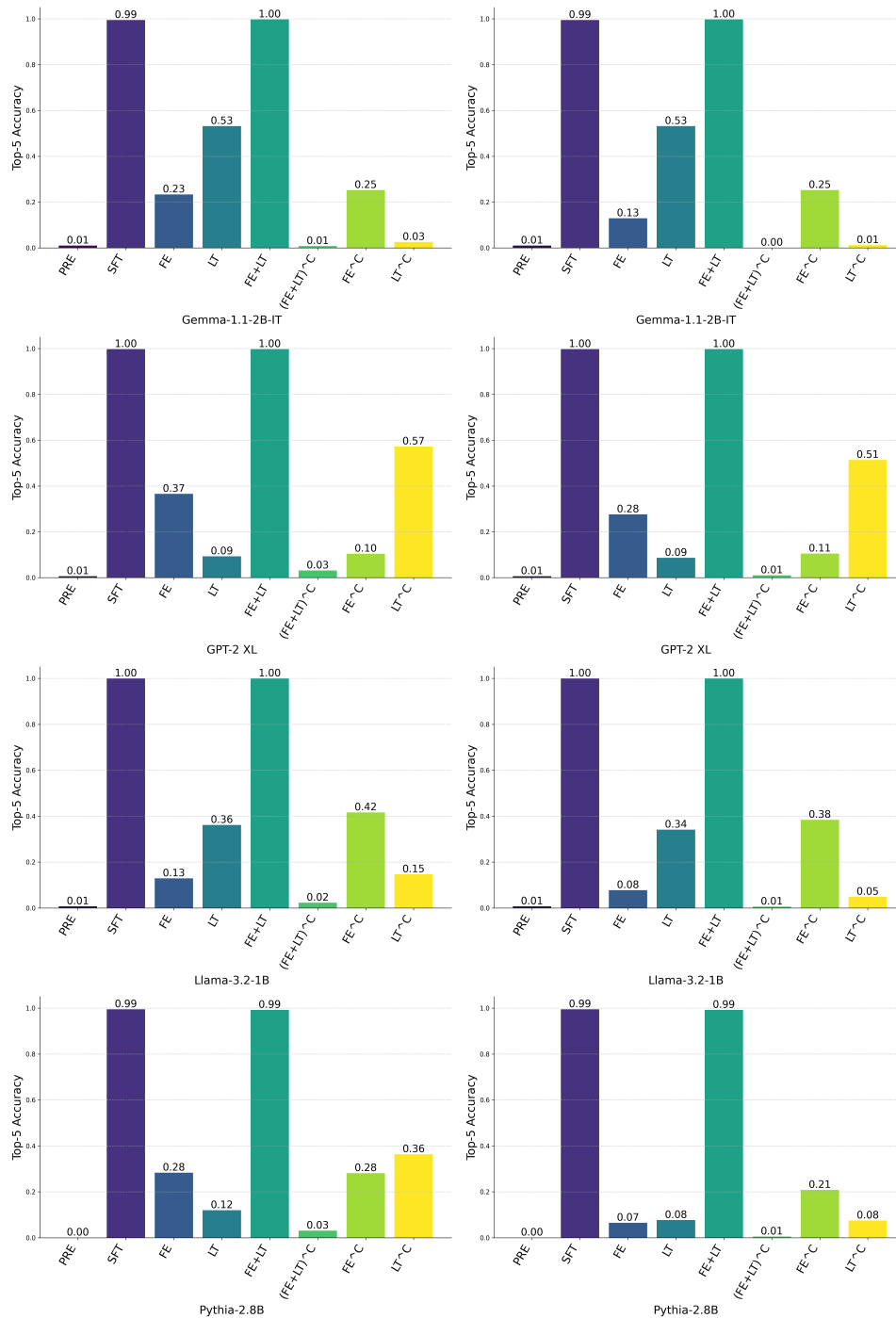


Figure 18: Top-5 accuracy–Sentence 1. Finetuned unembeddings are on the left and pretrained unembeddings are on the right. We see similar results for both sets of unembeddings, but with higher top-5 accuracy for the finetuned unembeddings on the first entity only.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports the less aggressive finetuning experiments (lower learning rate, zero weight decay, and supplemental training data from openwebtext and IMDB). The pattern behind Contribution 2 holds here too, with only weaker individual “extraction” and “recall” pathways.

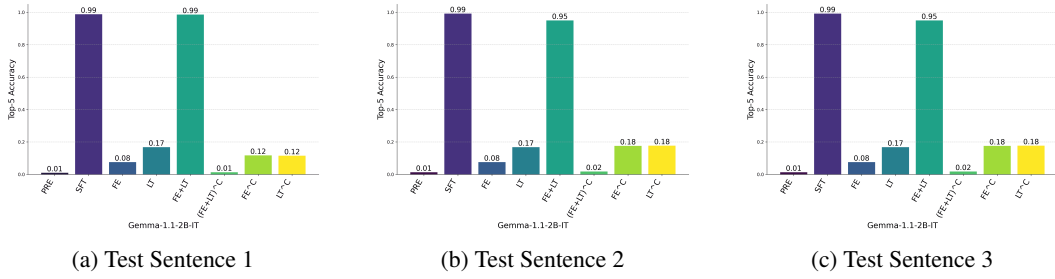


Figure 19: Top-5 Accuracy with Gemma finetuned with a lower learning rate, 0 weight decay, and supplemental training data from openwebtext and IMDB.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section reports baseline results for the component-grafting experiments (Contribution 3), grafting from SFT to pretrained models.

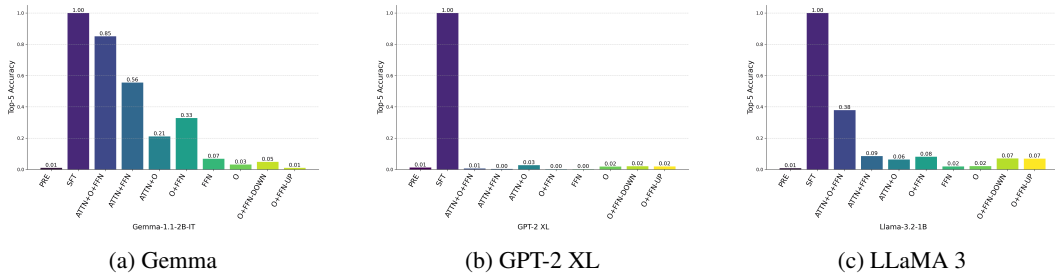


Figure 20: Top-5 Accuracy Component-Grafting Baselines — Sentence 1

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

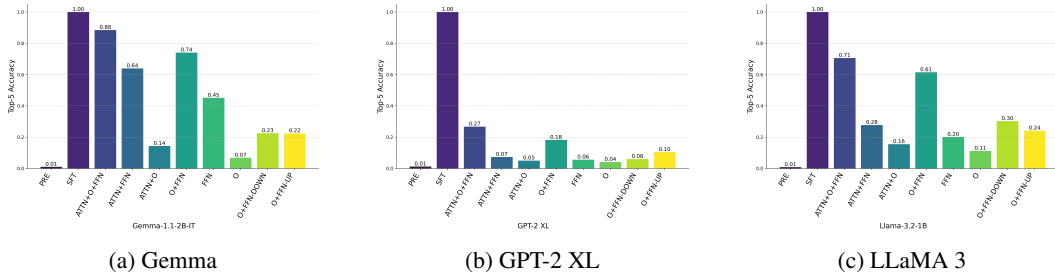


Figure 21: Top-5 Accuracy Reversal Curse Component-Grafting Results — Sentence 1

1620 C.10 TOKEN PROBABILITIES

1621
1622 We share five randomly sampled token-probability results showing the top 10 tokens for representative
1623 examples. Although we evaluate with top-5 accuracy, we display the top 10 tokens per example to
1624 give additional context.

1625 1626 C.10.1 LLAMA3: FE^C

1627
1628 Example 1:
1629 Target: Carolyn: 0.002
1630 J: 0.113
1631 L: 0.041
1632 Ch: 0.040
1633 Julie: 0.034
1634 Nicole: 0.024
1635 Nick: 0.023
1636 Michael: 0.023
1637 Jon: 0.023
1638 Jamie: 0.021
1639 Kelly: 0.017

1640 -----
1641 Example 2:
1642 Target: Lindsay: 0.024
1643 Michael: 0.093
1644 Ashley: 0.061
1645 Jennifer: 0.038
1646 Kim: 0.032
1647 Alexander: 0.025
1648 Lindsay: 0.024
1649 Milo: 0.023
1650 L: 0.021
1651 Jason: 0.018
1652 Lisa: 0.018

1653 -----
1654 Example 3:
1655 Target: Gwen: 0.015
1656 Michael: 0.030
1657 Paul: 0.028
1658 Julie: 0.024
1659 Elizabeth: 0.020
1660 Mary: 0.020
1661 S: 0.019
1662 E: 0.018
1663 J: 0.017
1664 L: 0.017
1665 A: 0.017

1666 -----
1667 Example 4:
1668 Target: Dominic: 0.006
1669 Is: 0.039
1670 Sarah: 0.031
1671 Jason: 0.026
1672 D: 0.026
1673 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1674 Mark: 0.019
1675

1676 -----
1677

1678 Example 5:
1679 Target: Ch: 0.045
1680 Ch: 0.045
1681 David: 0.024
1682 John: 0.022
1683 Peter: 0.018
1684 L: 0.018
1685 Mark: 0.018
1686 Michael: 0.017
1687 Richard: 0.016
1688 Ben: 0.012
1689 James: 0.012
1690 -----

1691 C.10.2 GEMMA: LT

1692
1693 Example 1:
1694 Target: Elizabeth: 0.040
1695 Julie: 0.495
1696 John: 0.130
1697 Stephen: 0.076
1698 Elizabeth: 0.040
1699 Hilary: 0.021
1700 Laurel: 0.018
1701 Tyne: 0.017
1702 Marian: 0.014
1703 Victoria: 0.013
1704 Juliet: 0.008
1705 -----

1706 Example 2:
1707 Target: Uta: 0.008
1708 John: 0.901
1709 Joan: 0.022
1710 Sally: 0.011
1711 Chelsea: 0.009
1712 Uta: 0.008
1713 Tyne: 0.003
1714 Gina: 0.002
1715 Elle: 0.001
1716 Lisa: 0.001
1717 Rosemary: 0.001
1718 -----

1719 Example 3:
1720 Target: Jennifer: 0.706
1721 Jennifer: 0.706
1722 John: 0.094
1723 Il: 0.028
1724 Me: 0.020
1725 Stephen: 0.015
1726 Carol: 0.012
1727 S: 0.008
David: 0.008
Elle: 0.007
T: 0.006

1728 -----
1729
1730 Example 4:
1731 Target: Marcia: 0.017
1732 John: 0.152
1733 Gwen: 0.136
1734 Susan: 0.050
1735 Tim: 0.045
1736 Ch: 0.041
1737 Celeste: 0.037
1738 Tara: 0.030
1739 T: 0.028
1740 Elle: 0.028
1741 Brittany: 0.026

1742 -----
1743 Example 5:
1744 Target: Heather: 0.035
1745 Ly: 0.916
1746 Heather: 0.035
1747 Hilary: 0.011
1748 Stephen: 0.007
1749 Julie: 0.002
1750 Rosemary: 0.002
1751 Ch: 0.002
1752 Wanda: 0.001
1753 Chelsea: 0.001
1754 Tem: 0.001
1755 -----

1756 C.10.3 GPT-2 XL: (FE+LT)^C

1757
1758 Example 1:
1759 Target: Fre: 0.000
1760 her: 0.037
1761 John: 0.018
1762 Jason: 0.017
1763 Chris: 0.016
1764 Adam: 0.015
1765 Zach: 0.014
1766 Josh: 0.013
1767 the: 0.013
1768 Michael: 0.013
1769 Ben: 0.012
1770 -----

1771 Example 2:
1772 Target: A: 0.002
1773 her: 0.043
1774 the: 0.017
1775 John: 0.016
1776 Tom: 0.016
1777 Peter: 0.012
1778 fellow: 0.012
1779 Michael: 0.011
1780 David: 0.011
1781 James: 0.011
1782 Jack: 0.010
1783 -----

```
1782 Example 3:
1783 Target: Matthew: 0.003
1784 his: 0.057
1785 the: 0.017
1786 Jennifer: 0.014
1787 fellow: 0.012
1788 Chris: 0.011
1789 Michael: 0.011
1790 Jason: 0.009
1791 John: 0.008
1792 James: 0.008
1793 Jessica: 0.007
```

```
1794 -----
1795 Example 4:
1796 Target: Cher: 0.000
1797 her: 0.042
1798 Tom: 0.026
1799 Robert: 0.019
1800 the: 0.018
1801 Matt: 0.018
1802 Michael: 0.016
1803 Brad: 0.016
1804 Chris: 0.014
1805 Johnny: 0.014
1806 John: 0.014
```

```
1807 -----
1808 Example 5:
1809 Target: Timothy: 0.001
1810 his: 0.056
1811 the: 0.025
1812 Michael: 0.013
1813 fellow: 0.013
1814 Robert: 0.013
1815 John: 0.012
1816 James: 0.010
1817 Tom: 0.010
1818 Mark: 0.008
1819 Peter: 0.008
1820 -----
```

1821 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1822 A characteristic result (for Gemma grafted on the first entity token) shows the model placing high
1823 probability on common tokens while still keeping the target token in the top 5.
1824

```
1825 Target: Robert: 0.016
1826 a: 0.365
1827 an: 0.151
1828 the: 0.091
1829 Robert: 0.016
1830 two: 0.009
1831 his: 0.008
1832 other: 0.006
1833 John: 0.005
1834 Morgan: 0.005
1835 my: 0.005
```